



AWS Academy Cloud Developing
Module 02 Student Guide
Version 2.0.6

200-ACCDEV-20-EN-SG

© 2024, Amazon Web Services, Inc. or its affiliates. All rights reserved.

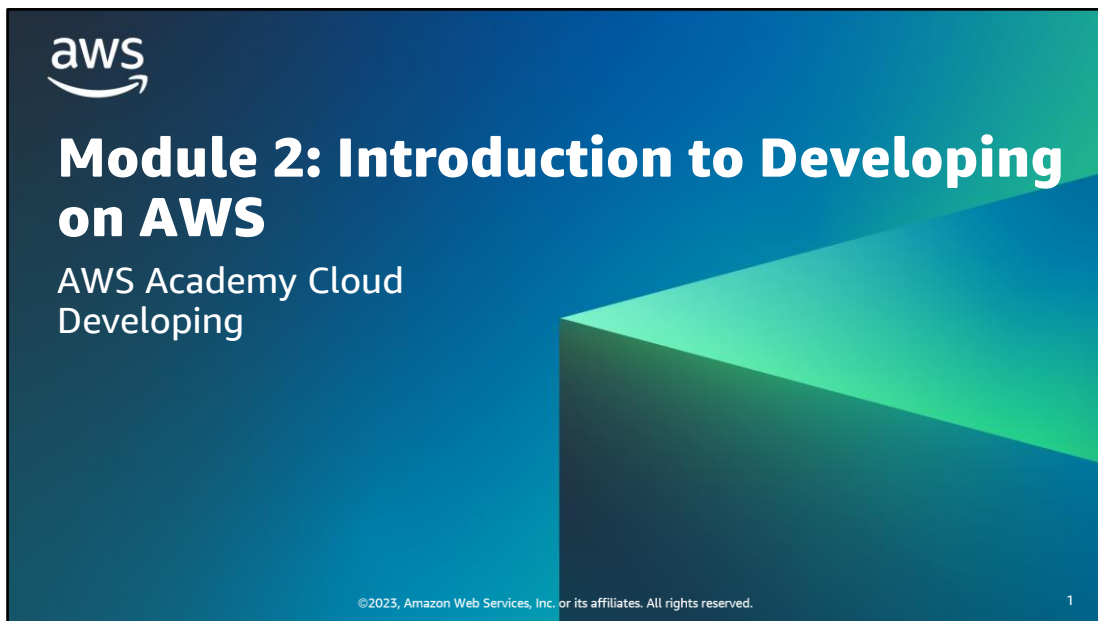
This work may not be reproduced or redistributed, in whole or in part,
without prior written permission from Amazon Web Services, Inc.
Commercial copying, lending, or selling is prohibited.

All trademarks are the property of their owners.

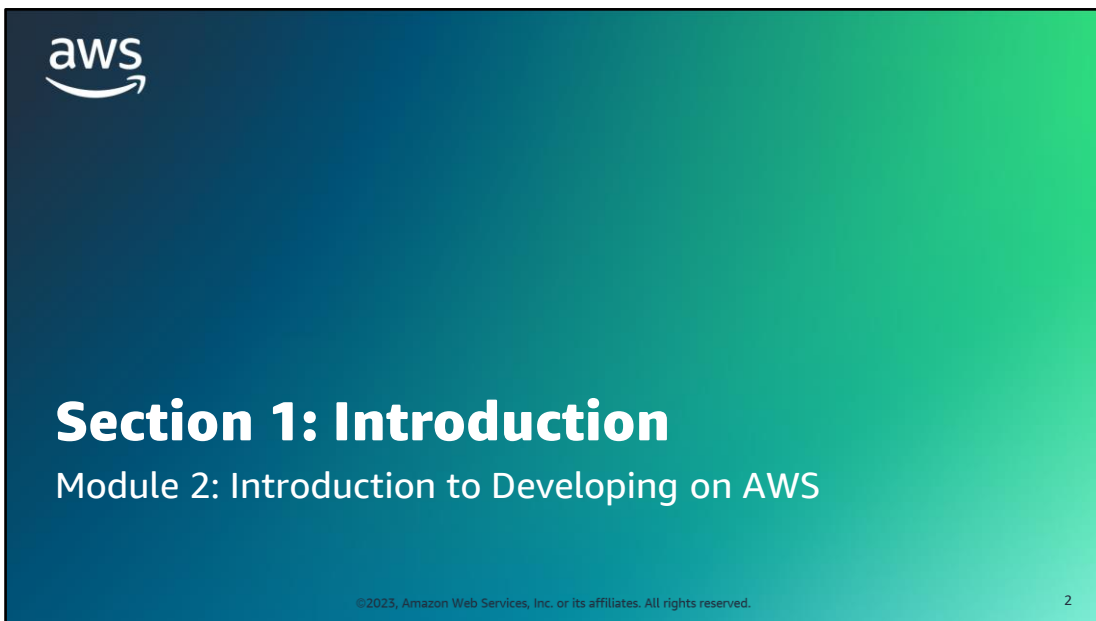
Contents

[Module 2: Introduction to Developing on AWS](#)

4



Welcome to Module 2: Introduction to Developing on AWS



Section 1: Introduction.

Module objectives

At the end of this module, you should be able to:

- Recognize the systems development lifecycle (SDLC).
- Describe how to start developing on AWS.
- Indicate how to use AWS SDKs.
- Identify the benefits of using Amazon Q Developer.



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

3

At the end of this module, you should be able to:

- Recognize the systems development lifecycle (SDLC).
- Describe how to start developing on AWS.
- Indicate how to use AWS SDKs.
- Identify the benefits of using Amazon Q Developer.

Module overview

Sections

1. Introduction
2. Systems development lifecycle
3. Steps to get started developing on AWS
4. Fundamentals of working with the AWS SDKs
5. Introduction to Amazon Q Developer

Demonstration

Installing the AWS CLI



Lab

Exploring AWS CloudShell and an IDE



Knowledge check

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

4

This module includes the following sections:

1. Introduction
2. Systems development lifecycle
3. Steps to get started developing on AWS
4. Fundamentals of working with the AWS SDKs
5. Introduction to Amazon Q Developer

This module also includes:

- A demonstration of installing the AWS CLI
- A lab where you learn how to work with AWS CloudShell and an IDE

Finally, you will complete a knowledge check to test your understanding of key concepts covered in this module.

Café business requirement

Sofía wants to start developing a web presence for the café. Before she starts coding, she wants to decide on a development environment to use to develop and run her code.

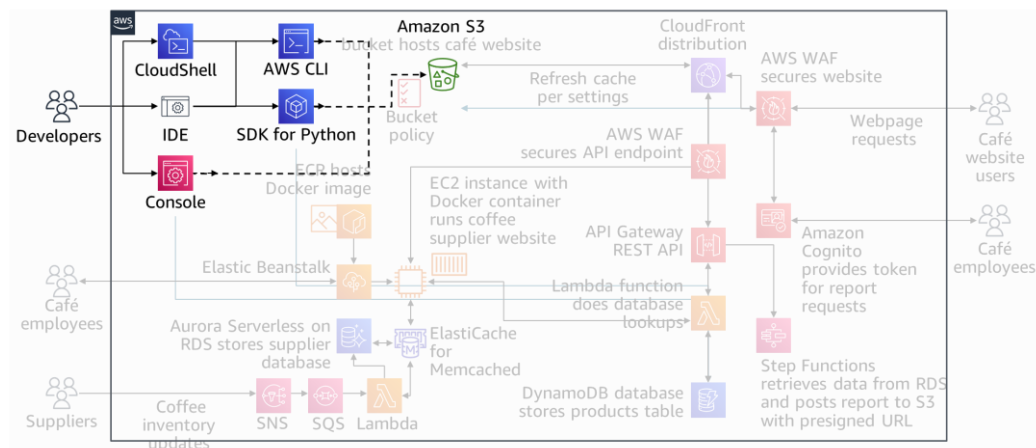


©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

5

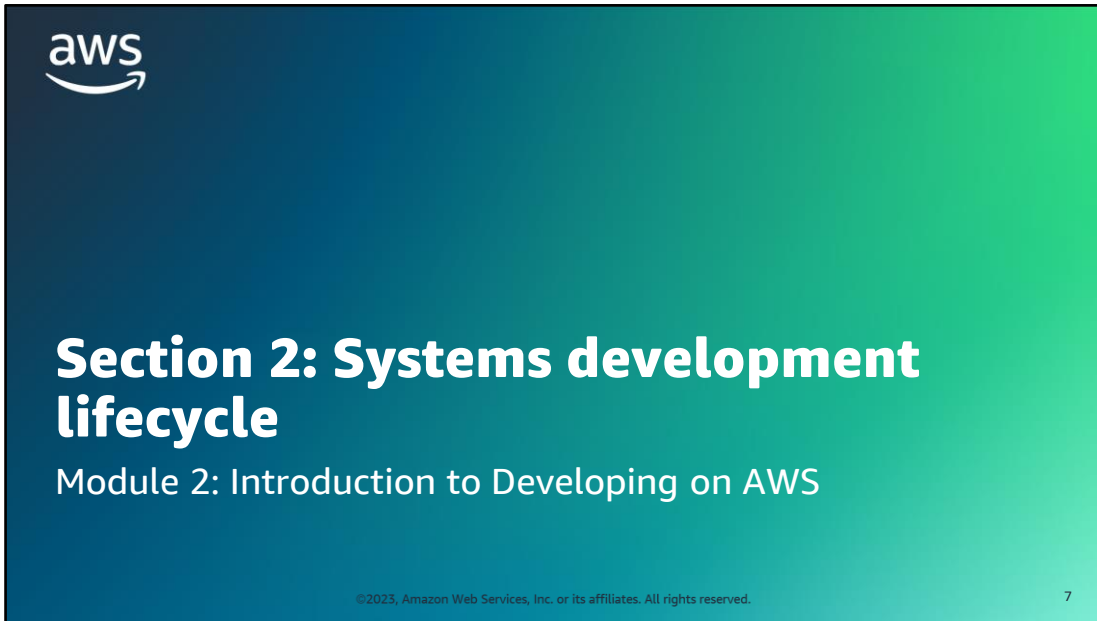
Sofía wants to start developing a web presence for the café. Sofía has Python development skills, and she is learning more about how to develop solutions in the cloud. Before she starts coding, she wants to decide on a development environment to use to develop and run her code.

Cloud development tools as part of developing a cloud application



The diagram on this slide gives an overview of the application that you will build through the labs in this course. The highlighted portions are relevant to this module.

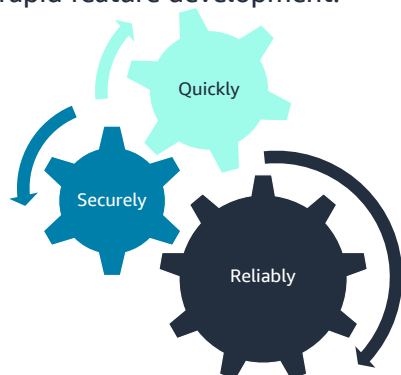
As highlighted in the diagram, you can use the AWS Management Console to launch AWS CloudShell. With CloudShell, you can use the AWS Command Line Interface (AWS CLI) or AWS SDK for Python (Boto3) to interact with AWS resources (for example, an S3 bucket).



Section 2: Systems development lifecycle.

The challenge of software delivery

Enterprises must bridge the gap between the stability of their operations and rapid feature development.



Deliver software with **zero tolerance** for outages:

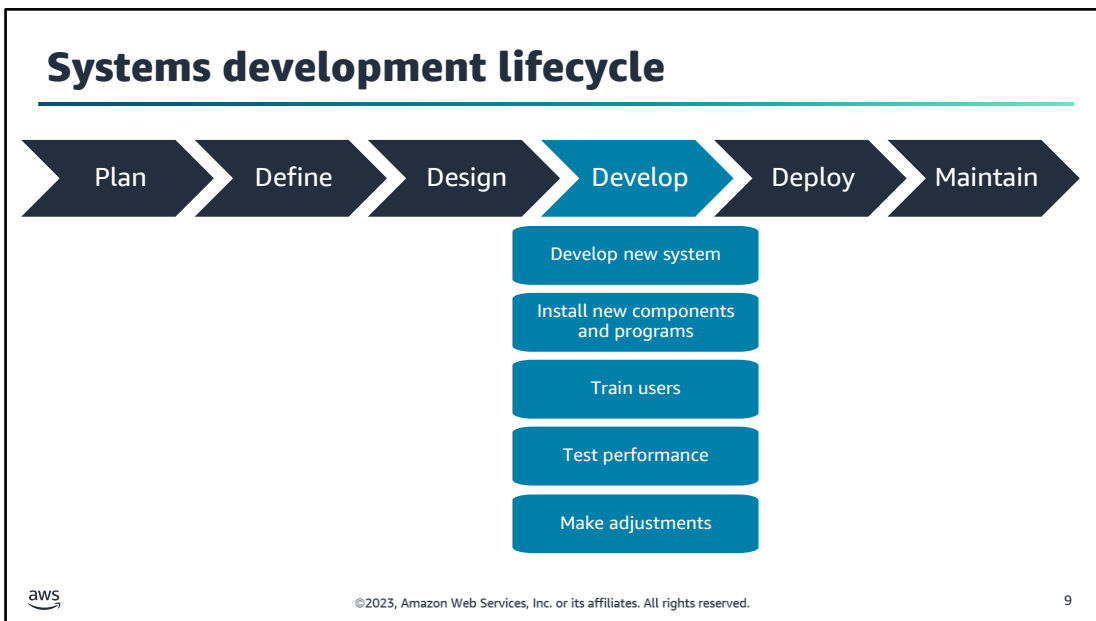
- Quickly
- Securely
- Reliably



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

8

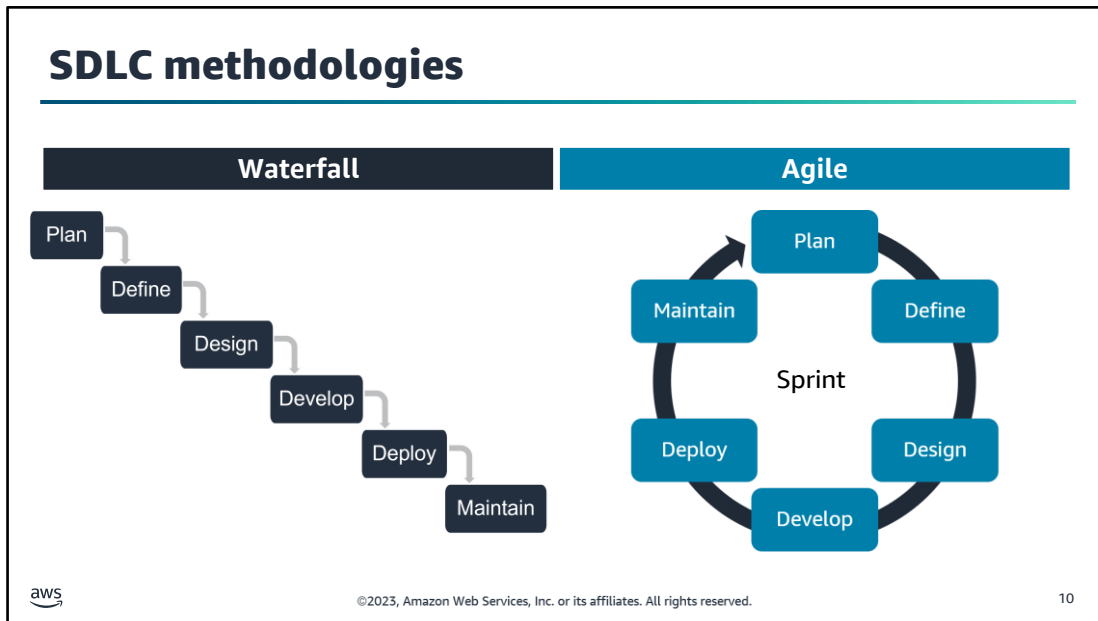
Software development and delivery is a challenge. Developers must quickly build solutions that are secure, reliable, and can scale to meet customer demands. All these goals must be accomplished while ensuring the security and integrity of the system. Methodologies are needed to bridge the gap between operational stability and rapid feature development.



Software development goes through a series of phases in the systems development lifecycle (SDLC). The SDLC is a conceptual model that is used in project management. It describes the stages in an information system development project, from an initial feasibility study to the maintenance of the completed application.

In general, an SDLC methodology follows these steps: plan, define, design, develop, deploy and maintain. This course will primarily focus on the Develop phase of SDLC. This course will briefly touch on the Deploy phase as well.

In the Develop phase, the new system is developed. New components and programs must be obtained and installed. System users must be trained, and all aspects of the system's performance must be tested. If necessary, bugs must be fixed and adjustments must be made to improve performance.

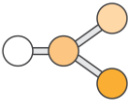


Various SDLC methodologies have been developed. Two of the most common methods are *waterfall* and *agile*.

- **Waterfall (or traditional) methodology** – This methodology is often considered to be the classic approach to the SDLC. The waterfall model describes a sequential development method. Each development phase has distinct goals and tasks that must be completed before the next phase can begin. Under this paradigm, product teams might not hear back from customers for months, and often not until the product is commercialized.
- **Agile software development methodology** – Agile is a newer conceptual framework that supports fast-paced, iterative software development. Under this newer paradigm, product teams push their work to customers as quickly as possible so that they can collect feedback and improve the previous iteration of their products. Concepts such as minimum viable product (MVP), release candidate, velocity, and others are all derived from the agile approach. There are various agile software development methodologies, such as Crystal methods, Dynamic Systems Development Method (DSDM), and Scrum. Under agile, software is developed iteratively in short time periods that are called *sprints*, which typically last 1–4 weeks.

When you develop applications in an agile way, your requirements might change frequently. Developing on AWS enables you to accommodate the changes and requirements more efficiently.

Major phases of software development

Develop			Deploy	Maintain
Code	Build	Test		
- Check in source code, such as .java files - Peer-review new code	- Compile code - Perform unit tests - Run style checkers - Gather code metrics - Create container images	- Test integration with other systems - Perform load tests - Test user interface (UI) - Perform penetration tests	Deploy code to production environments	Monitor code in production to quickly detect unusual activity or errors
				

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

11

The five major phases of software development are: Code, Build, Test, Deploy, and Maintain. Note that Source, Build, and Test fall under the Develop stage of the SDLC. Each phase provides increased confidence that the code will work in the way it's intended when it's eventually released to customers. The following overview lists the activities that occur in each phase:

- During the *Code phase*, developers write application source code and check changes into a source code repository, such as a Git repository. Many teams use code reviews to provide peer feedback of code quality before they ship code into production. Other teams use pair programming as a way to provide real-time peer feedback.
- During the *Build phase*, an application's source code is compiled and the quality of the code is tested on the build machine. The most common type of quality check is an automated test that doesn't require a server to run. These quality tests can be initiated from a test harness. Some teams extend their quality tests to include code metrics and style checks.
- During the *Test phase*, additional tests (that can't be done during the Build phase) are performed. These tests require the software to be deployed to a production-like environment. Often, these tests include integration testing with other live systems, load testing, user interface (UI) testing, and penetration testing. A common pattern is for engineers to deploy builds to a personal development stage, where they can check that their automated tests work correctly. They then deploy code to pre-production stages, where their application interacts with other systems to ensure that the software works in an integrated environment.
- Finally, code gets deployed to production. Though there are different deployment strategies, the common goals are to reduce risk when deploying new changes and to minimize the impact if a bad change is released to

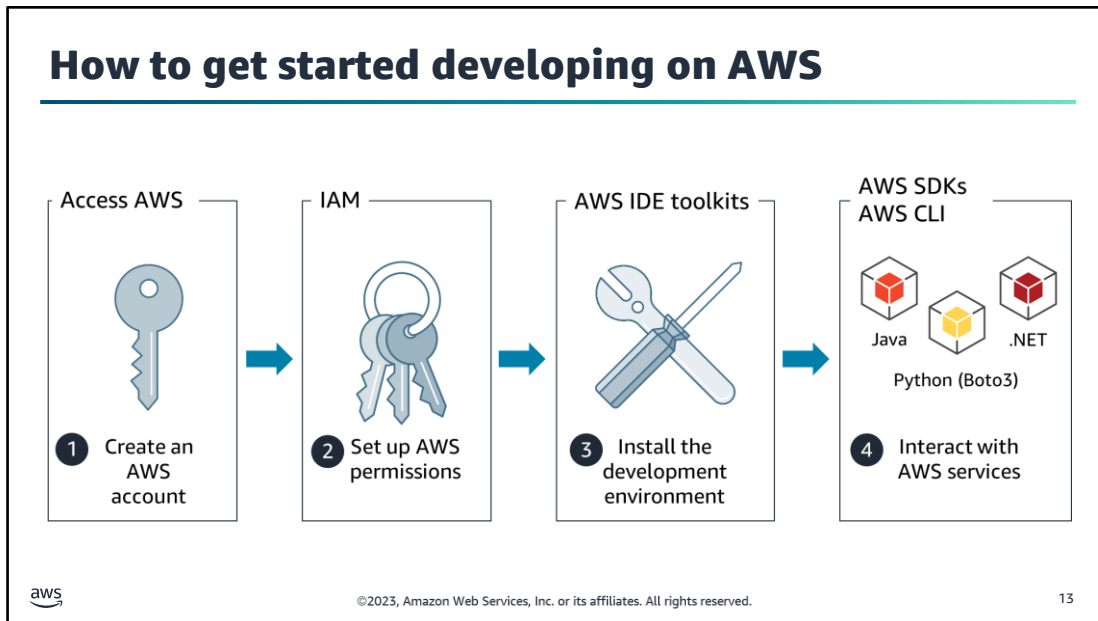
production.

- After code is deployed, it must be monitored in production to determine if everything works as expected.

Each of these steps can be automated without automating the entire release process.



Section 3: Steps to get started developing on AWS.



****For accessibility:** Step 1. Create an AWS account. Step 2. Set up AWS permissions with IAM. Step 3. Install the development environment with AWS IDE toolkits. Step 4. Interact with AWS services with AWS SDKs and AWS CLI. **End description.**

Before you start developing applications on AWS, you must do a few key things. First, you will need an *AWS account* and you must also create an *AWS Identity and Access Management (IAM) user*. This process includes setting up *AWS permissions* to access specific services and operations. Next, you must *install your development environment* for your language of choice. You must then install the AWS SDK for the language that you will use—or the AWS Command Line Interface (or AWS CLI)—so that you can *interact with AWS services*.

This section will discuss each AWS offering that’s involved in setting up AWS permissions, installing the development environment, and interacting with AWS services. Permissions will be covered in another module in this course.

Getting started: AWS account

The diagram illustrates the process of creating an AWS account. On the left, a vertical flowchart lists four steps: 1. Create an AWS account (highlighted with a dark blue circle), 2. Set up AWS permissions, 3. Install the development environment, and 4. Interact with AWS services. To the right, two screenshots of the AWS website are shown. The first screenshot, titled 'Create an AWS account', shows a form with fields for Email address, Password, Confirm password, and AWS account name, with a 'Continue' button. The second screenshot, titled 'Sign in', shows options for 'Root user' (selected) and 'IAM user', with a 'Next' button. Arrows indicate the flow from the first step to the 'Create an AWS account' form, and from the 'Sign in' form back to the 'Set up AWS permissions' step.

Create an AWS account

Go to aws.amazon.com to create an account

Choose [sign in to the console](#) if you already have an account

Sign in

Root user
Account owner that performs tasks requiring unrestricted access. [Learn more](#)

IAM user
User within an account that performs daily tasks. [Learn more](#)

Root user email address
username@example.com

Next

Continue

Sign in to an existing AWS account

© 2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

14

The steps to create an AWS account are as follows:

1. Open the AWS homepage at <http://aws.amazon.com>.
2. Choose **Create an AWS Account**.
 - **Note:** If you signed into AWS recently with the same browser, choose **Sign in to the Console**. If **Create a new AWS account** isn't available, then choose **Sign in to a different account** and then choose **Create a new AWS account**.
3. Enter the requested information and then choose **Continue**.
4. Select if the account is personal or professional. For learning purposes, select **personal**.
 - **Note:** Both types of accounts (personal or professional) have the same features.
5. Enter either your personal information or company information to continue.
 - If you are creating a professional account, enter the company's phone number and an organizational email address. Configuring a professional account with a personal phone number and email address can make an account less secure.
 - Ensure that you can access the email address that you used. You will need to access this email address to receive the confirmation message that your account was created.
6. Read and accept the AWS Customer Agreement at <https://aws.amazon.com/agreement/>.
7. Finally, choose **Create Account and Continue**.

Getting started: Permissions

Create an AWS account

Set up AWS permissions

Install the development environment

Interact with AWS services



IAM

- **Identity (authentication):**
Who can use your AWS resources
- **Access management (authorization):**
What resources can they use and in what ways



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

15

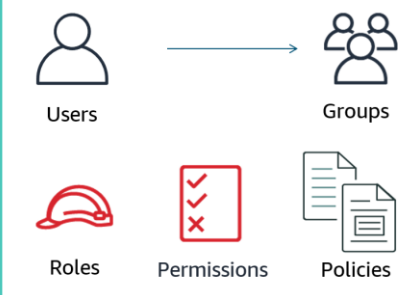
When first you create an AWS account, you begin with a single sign-on identity that has complete access to all AWS services and resources in the account. This identity is called the AWS account root user, and it's accessed by logging in with the email address and password that you used to create the account. A best practice is to not use the root account unless it's required. Instead, to increase the security of your AWS account, we recommend that you create an IAM user that will use access credentials instead of using your AWS account credentials.

IAM is a service that helps you securely control access to AWS resources for your users. You use IAM to control who can use your AWS resources (authentication) and what resources they can use and in what ways (authorization).

IAM enables you to create and manage users, groups, roles, and policies to control access to AWS services. IAM also enables identity federation between a corporate directory and AWS services.

IAM example

Create:



Example:

- **User** – John Doe (has AWS credentials).
- **Group** – John's IAM user belongs to the Developer group.
- **Role** – Trusted AWS entity with the Developer permissions policies.
- **Policy** – John's IAM role is associated with the Developer policy and has access to specific services and operations.



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

16

For example, consider a situation where you are responsible for an organization's users. You could use IAM to set up a user account for each developer in your organization. Each user has an IAM user account that they must use to access AWS services. You could then create an IAM role for developers and associate the developer role with each developer's user account. The developer role can be configured with policies that control which services and operations the role can access. Developers then assume the IAM role to have the permissions they need to work in the environment. When updates are needed to the permissions developers require, the IAM Policy associated with the IAM role is updated and any developer assuming the role gets the required permissions.

You will learn more about IAM in the next module.

Getting started: Development environment



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

17

IDEs and AWS IDE Toolkits:

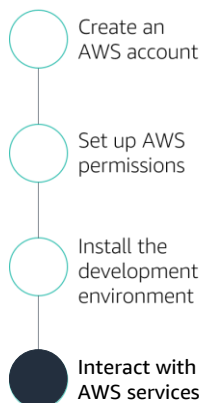
- Eclipse IDE (Eclipse Foundation)
- IntelliJ IDEA (JetBrains)
- PyCharm (JetBrains)
- Microsoft Visual Studio
- Microsoft Visual Studio Code
- Microsoft Azure DevOps
- Rider (JetBrains)

When you set up your development environment, you need to install your programming language of choice and an integrated development environment (IDE). AWS offers several language-specific IDEs for you to write, run, debug, and deploy applications.

For more details about available AWS SDKs, IDEs, and toolkits, refer to “Tools to Build on AWS” at <https://aws.amazon.com/tools/>.

For information about how to install your development environment, follow the instructions from the technology vendors of the platform and the IDE.

Getting started: AWS services



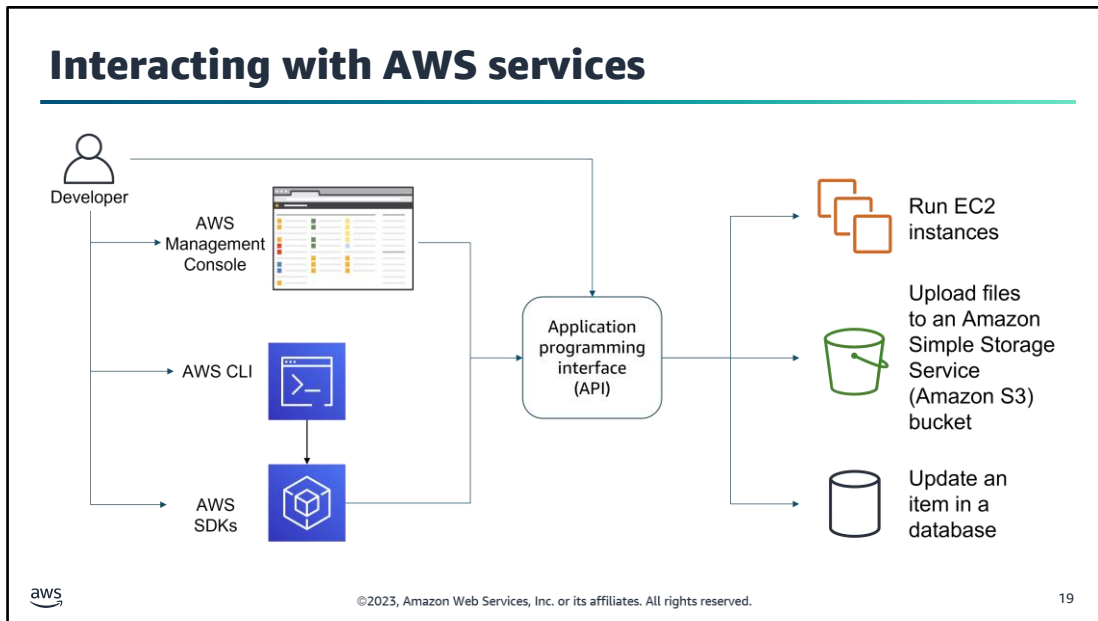
- AWS Management Console
- Programmatically
 - AWS Command Line Interface (AWS CLI)
 - AWS software development kits (SDKs)
 - Service application programming interfaces (APIs)



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

18

You can interact with AWS through the AWS Management Console, or you can interact with AWS programmatically. With programmatic access, services can be called through the AWS Command Line Interface (AWS CLI), the AWS software development kit (SDK) for the supported language, or the service application programming interface (API).



All AWS services are managed through a common, REST-like API. AWS provides an API for each of its services.

You can interact with the APIs to access your AWS resources in four ways:

- *Directly* – You can call the APIs directly.
- *AWS Management Console* – An implementation of the API calls in a web console. It provides a rich graphical interface to a majority of the features that are offered by AWS.
 - **Note:** Occasionally, new features might not be available in the console when the feature initially launches.
- *AWS Command Line Interface (AWS CLI)* – An open source tool that enables you to interact with AWS services by using commands in your command line interface.
- *AWS software development kits (AWS SDKs)* – Packages that enable you to access AWS services and resources in a variety of popular programming languages.

The AWS CLI and the SDKs provide flexibility. They enable you to create your own tools and customize existing AWS features. For example, you can create your own scripts or applications for launching EC2 instances that enforce using a specific set of Amazon Machine Images (AMIs), add a standard set of tags, and so on.

You can use these access modes interchangeably. For example, you can create server EC2 instance through an SDK call. This instance can be queried for its configuration information by using an AWS CLI command (`aws ec2 describe-instances`). Finally, you can terminate the server instance by using the console. (It might sometimes take a few seconds or a few minutes for changes made

through the AWS CLI or the API to display in the console.)

For information about the AWS SDKs and AWS CLI, see “Tools to Build on AWS” at <https://aws.amazon.com/tools/>.

Command line tools



AWS CLI
(Windows, macOS, Linux
or Unix)



AWS Tools for
PowerShell
(Windows, macOS, Linux)



AWS Serverless
Application Model
(AWS SAM) Local



AWS Amplify

[AWS Command Line Interface user guide, Installing the AWS CLI](#)



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

20

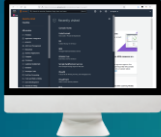
When you use the command line, you can use various tools to call AWS services:

- *AWS Command Line Interface (AWS CLI)* is an open source tool that enables you to interact with AWS services by using commands in your command line interface. With minimal configuration, you can start using the command prompt in your favorite terminal program to access functionality that's equivalent to the AWS Management Console. For information on the AWS CLI, see "AWS Command Line Interface" at <https://aws.amazon.com/cli/>.
- *AWS Tools for PowerShell* offers the AWS Tools for Windows PowerShell and AWS Tools for PowerShell Core. They are PowerShell modules that are built on the functionality that's exposed by the SDK for .NET. The AWS PowerShell Tools enable you to script operations on your AWS resources from the PowerShell command line. For more information, see "AWS Tools for PowerShell User Guide" at <https://docs.aws.amazon.com/powershell/latest/userguide/pstools-welcome.html>.
- *AWS Serverless Application Model (AWS SAM) Local* is an AWS CLI tool for the local development and testing of serverless applications. For information about AWS SAM Local, see "aws-sam-cli GitHub repository" at <https://github.com/aws-labs/aws-sam-cli>.
- *AWS Amplify* enables you to create, configure, and implement scalable mobile applications that are powered by AWS. AWS Amplify provisions and manages your mobile backend. It also provides a simple framework so that you can integrate your backend with your frontends, including iOS, Android, web, and React Native. AWS Amplify also automates the application release process of both your frontend and your backend, enabling you to deliver features faster. For more information, see "AWS Amplify" at <https://aws.amazon.com/amplify/>.

For more information, see:

- AWS command line tools – <https://aws.amazon.com/tools/>.
- How to get started with the AWS CLI – <https://aws.amazon.com/cli/>.
- How to install the AWS CLI – <https://docs.aws.amazon.com/cli/latest/userguide/cli-chap-install.html>.

Demonstration: Installing the AWS CLI





©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

21

Now, the educator might choose to demonstrate installing the AWS CLI.

AWS CLI: Command line format



AWS CLI

Service
(command) Operation
(sub-command)

Parameters

```
$ aws ec2 stop-instances --instance-id i-1234567890abcdef0
```

```
$ aws ec2 run-instances --cli-input-json file:///lcf/webserver.json
```

```
$ aws help
$ aws ec2 help
$ aws ec2 describe-instances help
```



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

22

Calls made from the AWS CLI have a certain format:

- The command line always begins with *aws*. This string is how you invoke the AWS CLI.
- The next component is a *service command*. This string is the top-level AWS service that you are calling (for example, Amazon EC2). For a list of currently supported services, see “Available services” at <https://docs.aws.amazon.com/cli/latest/reference/#available-services>.
- The *operation subcommand* is next. This string is the operation that you want to perform on that service. This example uses the run-instances subcommand to request the creation of a new EC2 instance.
- Then, you specify any *parameters*. The parameters are any arguments that are needed to perform the operation. Argument names are preceded by two dashes (--). For example, to create an EC2 instance, you must run the Amazon EC2 run-instances operation and pass in several parameters, such as the Amazon Machine Image (AMI) ID.
- You can also specify *options*. The options are choices that you can specify when you run an operation. For example, you can use the --query option to limit the response text so that it returns only the instance ID of your new instance.

AWS CloudShell



AWS CloudShell

AWS CloudShell provides CLI access in a web browser

- Available in the AWS Management Console
- Pre-authenticated session
- Pre-installed with the AWS CLI, SDKs, and other tools
- No cost
- 1 GB of storage per Region

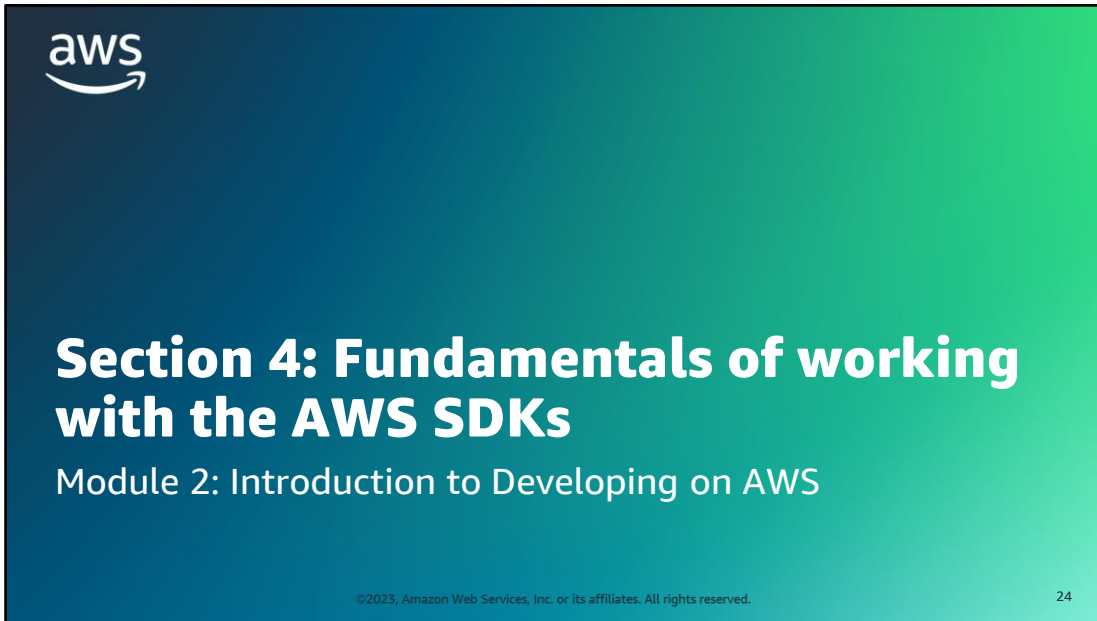


©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

23

Some of the features of AWS CloudShell include:

- CloudShell inherits your credentials from the AWS Management Console. This means there are not key files to manage when using the AWS CLI.
- It runs on an Amazon Linux 2 instance that's fully managed. All the latest versions of the tools are installed without needing to update or patch the environment
- It incurs no cost. CloudShell includes 1 GB of persistent storage per Region. You only pay for the resources that you create and run on AWS.
- It's customizable (per Region) for storing scripts, files, configuration preferences, and tools.



Section 4: Fundamentals of working with AWS SDKs.

AWS SDKs



AWS SDK

AWS SDK for:

- C++
- Go
- Java
- JavaScript in the browser
- JavaScript in Node.js
- .NET and Xamarin
- PHP
- Python (Boto3)
- Ruby

AWS Mobile SDK for:

- Android
- iOS
- .NET and Xamarin
- Unity

AWS IoT Device SDK for:

- Arduino Yún
- C++
- Embedded C
- Java
- JavaScript
- Python



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

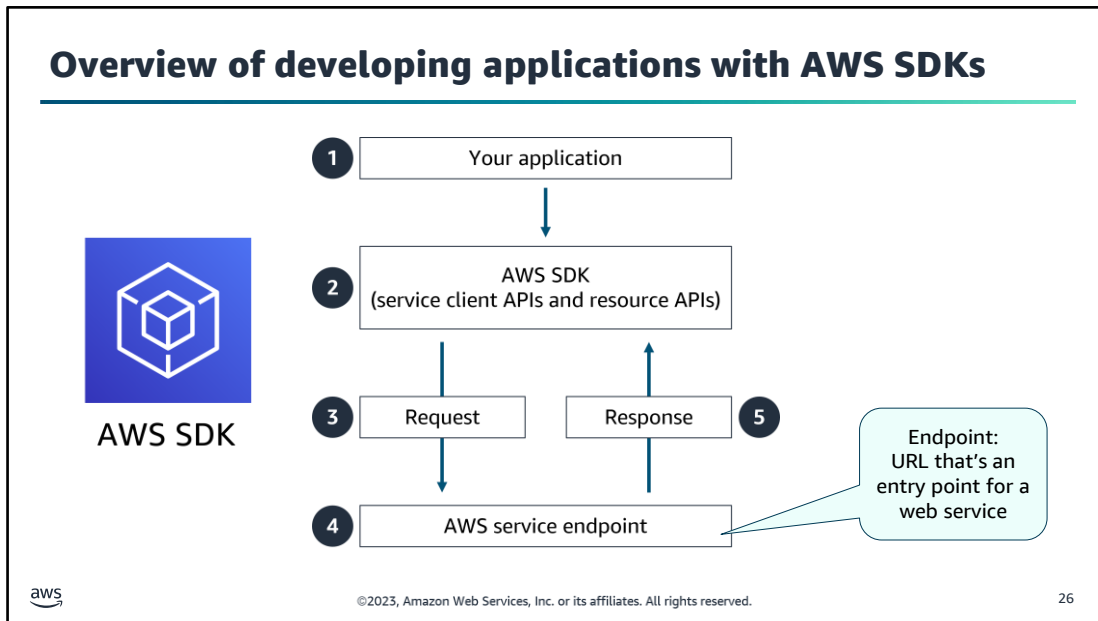
25

The AWS SDKs are available in various programming languages and technology platforms. You can use the AWS SDKs to call AWS services from your application code.

Say, for example, that you are developing a music-sharing application. Your application needs to upload a user's music files to an S3 bucket. In this case, your application could use the AWS SDK to programmatically upload files to the S3 bucket.

To learn more about the available AWS SDKs (including installation instructions), see:

- "SDKs" at <https://aws.amazon.com/tools/>.
- "AWS SDKs and Tools Reference Guide" at https://docs.aws.amazon.com/sdkref/latest/guide/overview.html?icmpid=docs_homepage_sdktoolkits.



~alt text group28: Steps for developing with AWS SDKs, described in notes.

When you develop applications on AWS, you can use the AWS SDKs to make API calls to AWS services and get responses back from those services. This diagram illustrates application development with the AWS SDKs.

1. You write an application using an AWS SDK in your language of choice.
2. Each AWS SDK provides one or more APIs for working with AWS services.
3. The AWS SDK constructs an HTTP(S) request for use with the service API, and sends the request to the AWS service endpoint.
4. The service performs the request and sends a response.
5. The AWS SDK processes the response and propagates it back to your application.

Service client API

- Has one method per service operation
- Has objects for request and result data
- Example (in Python):

```
# List Objects in bucket using the client API
# Return type: dict / additional API calls needed to get objects
def listClient():
    s3client = boto3.client('s3')
    response = s3client.list_objects_v2(Bucket='DOC-EXAMPLE-BUCKET')
    for content in response['Contents']:
        print(content['Key'], content['LastModified'])
    return
```



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

27

AWS SDKs offer two levels of APIs that you can use to call AWS services: *service client APIs* and *resource APIs*.

Service client API

AWS SDKs provide a set of *client classes*. Each client class exposes a direct mapping of the API for the AWS service. These client objects have a method for each operation that the service supports, with corresponding objects that represent the request parameters and the response data. Using this *low-level* client API gives you full control over the requests that you make to the service, which enables you to tightly control the behavior and performance of your calls to AWS services.

For example, both the SDK for Java and the SDK for .NET and Xamarin have `Amazon<Service>Client` classes that correspond to each service, such as `AmazonS3Client` and `AmazonDynamoDBClient`.

This example compares how to list the objects in the `DOC-EXAMPLE-BUCKET` S3 bucket by using the SDK for Python service client API.

For more information, see “S3: Client” at <https://boto3.amazonaws.com/v1/documentation/api/latest/reference/services/s3.html#client> of the SDK for Python documentation.

Resource API

- Has one class per conceptual resource
- Defines service resources and individual resources
- Example (in Python):

```
# List Objects in Bucket using the Resource API
# Resources represent an object-oriented interface to Amazon Web Services (AWS).
# They provide a higher-level abstraction than the raw, low-level calls made by
# service clients
def listResource():
    s3resource = boto3.resource('s3')
    bucket = s3resource.Bucket('DOC-EXAMPLE-BUCKET')
    for object in bucket.objects.all():
        print(object.key, object.last_modified)
    return
```



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

28

Some AWS SDKs, such as the AWS SDK for Python (Boto3), provide higher-level APIs that are called *resource APIs*. Resource APIs provide a higher-level abstraction than the low-level calls that are made by clients. Instead of a single client object exposing the entire API for a service, the resource APIs consist of classes that represent each of the conceptual resources that you interact with when you use a service. These classes expose methods to retrieve data about the resource, invoke actions that can be taken on the resource, and retrieve links to other related resources. Access to the resources are then provided through objects and collections. Resources include service such as Amazon S3 or Amazon Simple Queue Service.

For example, the Amazon S3 service resource exposes methods to perform actions on the Amazon S3 service. An individual Amazon S3 resource (such as an object) has an identifier or object key, and attributes (such as `last_modified`).

This example compares how to list the objects in the *DOC-EXAMPLE-BUCKET* S3 bucket by using the resource API for the SDK for Python.

For more information about resource APIs, see in the SDK for Python for documentation:

- “Resources” at <https://boto3.amazonaws.com/v1/documentation/api/latest/guide/resources.html>.
- “Service Resource” at <https://boto3.amazonaws.com/v1/documentation/api/latest/reference/services/s3.html#service-resource>.

Specifying AWS Regions

Specify Region when you instantiate the service client.

```
dynamodb_client =  
boto3.client('dynamodb',  
region_name='us-east-1')
```

Default Region is specified in AWS config.

```
~/.aws/config  
[default]  
region=us-west-2  
output=json
```



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

29

The AWS Cloud infrastructure is built around Regions and Availability Zones. A Region is a physical location in the world that has multiple Availability Zones. When you write your applications, you can access AWS services that physically reside in a specific geographic region. How you specify the AWS Region depends on the AWS SDK that you use.

With some AWS SDKs, such as the AWS SDKs for Java and .NET, you can specify the Region when you instantiate the service client. You must create a separate instance of the service client for each Region that you want to work with. For other SDKs, such as the AWS SDK for Python (Boto3), you can use the default Region that is set in the `~/.aws/config` file.

For more information about specifying AWS Regions with the AWS SDKs, see:

- “Developer Guide – AWS SDK for Java 2.x” at <https://docs.aws.amazon.com/AWSSdkDocsJava/latest/DeveloperGuide/java-dg-region-selection.html>.
- “Configure the AWS Region” SDKs for .NET and Xamarin at <https://docs.aws.amazon.com/sdk-for-net/v3/developer-guide/net-dg-region-selection.html>.
- “Configuration” for the SDKs for Python at <https://boto3.amazonaws.com/v1/documentation/api/latest/guide/quickstart.html#configuration>.
- “Setting the AWS Region” SDK for JavaScript at <https://docs.aws.amazon.com/sdk-for-javascript/v2/developer-guide/setting-region.html>.
- “Configure the AWS SDK for Ruby” at <https://docs.aws.amazon.com/sdk-for-ruby/v3/developer-guide/setup-config.html>.

Handling exceptions

Error response example

```
<?xml version="1.0" encoding="UTF-8"?>
<Error>
  <Code>NoSuchKey</Code>
  <Message>The resource you requested does not exist</Message>
  <Resource>/DOC-EXAMPLE-BUCKET/myfoto.jpg</Resource>
  <RequestId>4442587FB7D0A2F9</RequestId>
</Error>
```

What to do?

- **HTTP 400 series** (client error): handle error in application
- **HTTP 500 series** (server error): retry operation

Error response element descriptions

Error: Container for all error elements.

Code: String that uniquely identifies an error condition.

Message: Generic description of the error code in English.

Resource: The resource that's involved in the error.

RequestId: ID of the request that's associated with the error.



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

30

When an API call is made to an AWS service, the service performs the request and returns a response that includes an HTTP status code. If the request is successful, the AWS service returns an *HTTP 200* response code. If the request is unsuccessful, the AWS service returns an *error response*, which consists of the following elements:

- *Error* – Container for all error elements.
- *Code* – String that uniquely identifies an error condition. It's meant to be read and understood by programs that detect and handle errors by type.
- *Message* – Generic description of the error condition in English. It's intended for a human audience.
- *Resource* – Resource that's involved in the error.
- *RequestId* – ID of the request that's associated with the error.

You can use the error code to determine how to handle the error.

- A *400 series error* means that you must handle the error in your application. For example, Amazon S3 will return a 404 error code if the bucket you are trying to access doesn't exist. Your application can handle this error by first creating the bucket, and then performing operations on it.
- A *500 series error* indicates an internal server error. In this case, you could retry the operation. Each AWS SDK implements automatic retry logic. You can configure the maximum number of retry attempts. In addition to simple retries, each AWS SDK implements an exponential backoff algorithm to retry after failed connection attempts. With an exponential backoff algorithm, you specify progressively longer waits after each failed attempt before retrying your request.

For more information, see:

- Error Handling section of the Developer Guide for specific AWS services
- “Retry behavior” at <https://docs.aws.amazon.com/general/latest/gr/api-retries.html>.

Exceptions



SDK for Java
SDK for .NET and Xamarin
AmazonServiceException
(or subclass)
AmazonClientException



SDK for Python
Botocore.exceptions.ClientError



SDK for JavaScript
Asynchronous callback
function(error, data) { ... }



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

31

AWS SDKs throw exceptions to communicate errors in different ways.

SDK for Java

The AWS SDK for Java throws the following unchecked (runtime) exceptions when errors occur:

- **AmazonServiceException** (and subclasses) – Indicates that the request was correctly transmitted to the service. However, the service couldn't process it, and it returned an error response instead. The exception has the following information that you can use to determine how your application should handle the error:
 - HTTP status code returned by the service, such as *404 Not Found*
 - AWS error code returned by the service, such as *NoSuchBucket*
 - Detailed error message from the service, such as *The specified bucket does not exist*
 - AWS request ID for the failed request
- **AmazonClientException** – Indicates that a problem occurred inside the Java client code. The problem might have occurred either while trying to send a request to AWS or while trying to parse a response from AWS. For example, the SDK for Java will throw an **AmazonClientException** if no network connection is available when you try to call an operation on one of the clients.
- **java.lang.IllegalArgumentException** – This exception is thrown if you pass an illegal argument when you perform an operation on a service.

For more information, see “Developer Guide - AWS SDK for Java 2.x” at <https://docs.aws.amazon.com/AWSSdkDocsJava/latest/DeveloperGuide/java-dg-region-selection.html>.

SDK for .NET and Xamarin

The SDK for .NET and Xamarin throws `Amazon.Runtime.AmazonServiceException` and `Amazon.Runtime.AmazonClientException`, which are similar to the errors from the SDK for Java.

SDK for Python

The SDK for Python throws `botocore.exceptions.ClientError`.

SDK for JavaScript

Each service method that sends a request can accept a callback as the last parameter with the signature `function(error, data) { ... }`. This callback is called when the response or error data is available.

For more information, see “Calling Services Asynchronously” at <https://docs.aws.amazon.com/sdk-for-javascript/v2/developer-guide/calling-services-asynchronously.html> in the SDK for JavaScript Developer Guide.

Lab 2.1: Exploring AWS CloudShell and an IDE





©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

32

You will now complete Lab 2.1: Exploring AWS CloudShell and an IDE.

Lab: Tasks

1. Exploring AWS CloudShell
2. Exploring an integrated development environment (IDE)



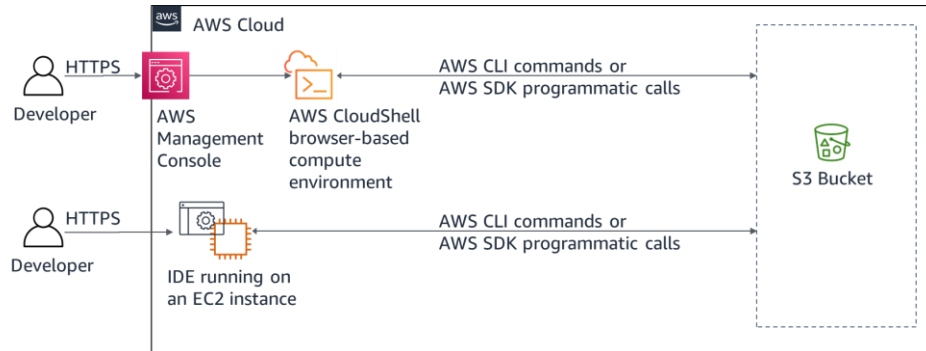
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

33

In this lab, you will complete the following tasks:

1. Exploring AWS CloudShell
2. Exploring an integrated development environment (IDE)

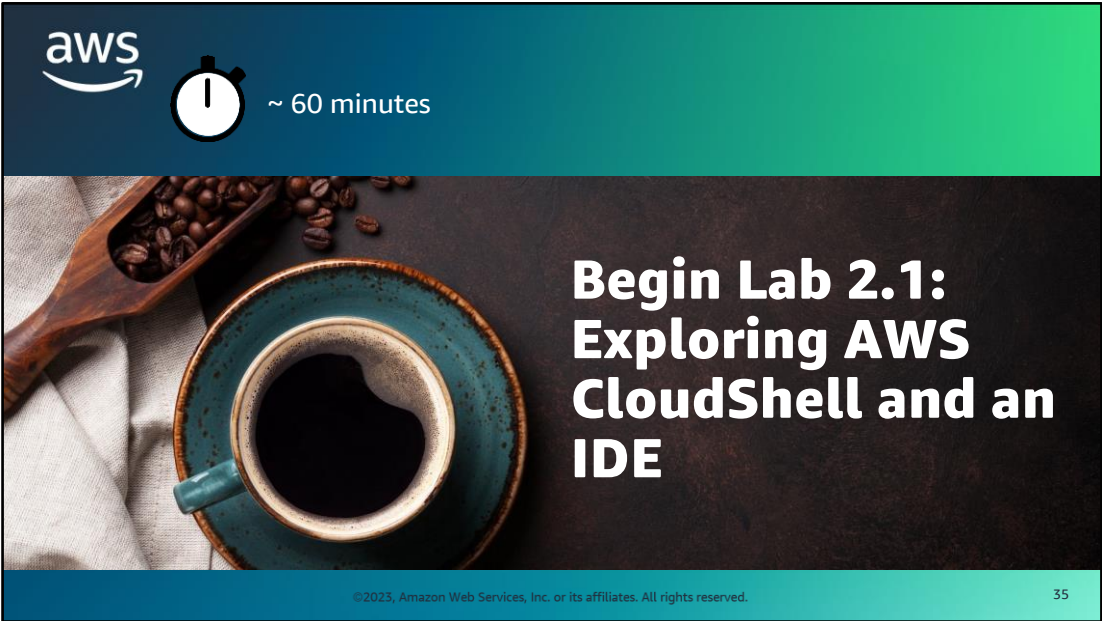
Lab: Final product



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

34

The diagram summarizes what you will have done after you complete the lab. You will have used the AWS Management Console and AWS CloudShell to interact with an Amazon S3 bucket.



The image is a slide for an AWS lab. At the top, there is a dark blue header with the AWS logo on the left, a clock icon in the center, and the text '~ 60 minutes' on the right. Below the header is a large image of a cup of coffee on a saucer, with coffee beans scattered around it. To the right of the coffee image, the text 'Begin Lab 2.1: Exploring AWS CloudShell and an IDE' is written in a large, bold, white font. At the bottom of the slide, there is a teal footer with the copyright text '©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.' on the left and the number '35' on the right.

aws ~ 60 minutes

**Begin Lab 2.1:
Exploring AWS
CloudShell and an
IDE**

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved. 35

It is now time to start the lab.

Lab debrief: Key takeaways

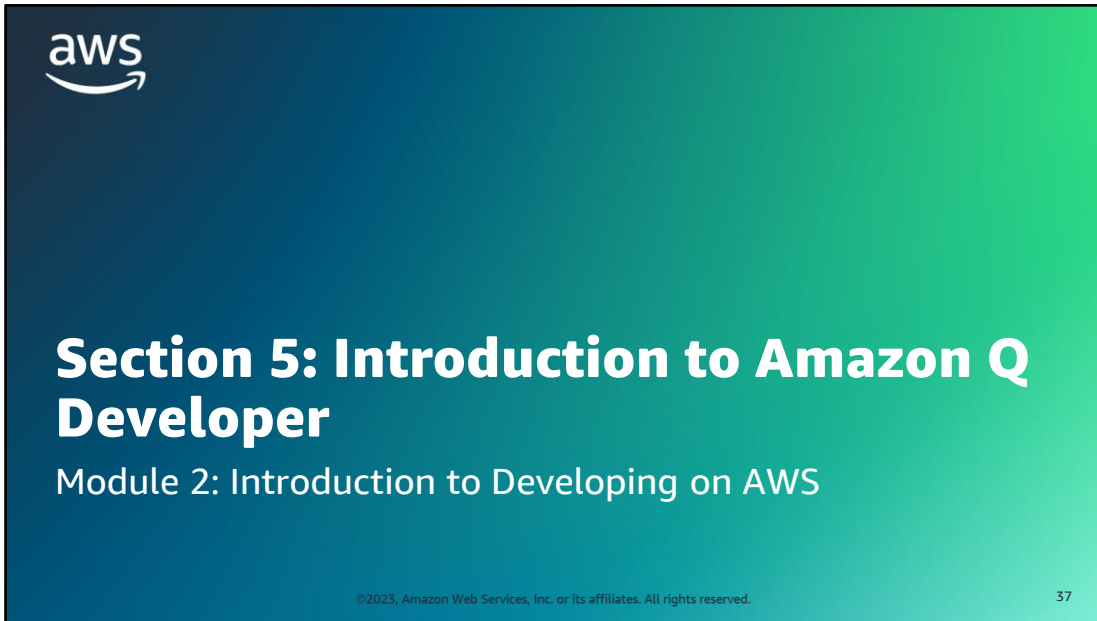


aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

36

Your educator might choose to lead a conversation about the key takeaways from this lab after you have completed it.



Section 5: Introduction to Amazon Q Developer

Amazon Q Developer



Amazon Q Developer

- Generative AI-powered coding assistant
- Designed for developers and IT professionals
- Generates code and helps you understand, build, extend, and operate AWS applications
- Scan code for security vulnerabilities
- Secure and private by design



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

38

To alleviate some of the challenges with writing code, AWS introduced Amazon Q Developer. Amazon Q Developer is a coding assistant that those without extensive experience can use. It is trained on years of high-quality AWS examples and documentation. It can also be trained on your company's code and systems. Amazon Q Developer can chat about code, provide in-line code completions, generate new code, and scan your code for security vulnerabilities. It can also make code upgrades and improvements, such as language updates, debugging, and optimizations. With Amazon Q Developer, you can describe the application you want to create in natural language, and Amazon Q Developer will generate and suggest code for the application. You can build applications without needing to write all code yourself. The no-code approach of Amazon Q Developer can be particularly useful for creating simple web applications, automating business processes, or building prototypes and proofs of concept.

For more information, see “Getting Started” in the *AWS Amazon Q Developer User Guide* on the content resources page of your online course.

Amazon Q Developer supports you across the software development lifecycle (SDLC)



Plan

- Ask questions and get referenceable and contextual guidance.
- Explain code with conversational coding.



Create

- Receive in-line code recommendations for multiple languages.
- Implement features through comments or code prompts.
- Chat in your integrated development environment (IDE).



Test and secure

- Generate unit tests.
- Scan project code for security vulnerabilities and get remediation suggestions.



Operate

- Troubleshoot errors.
- Troubleshoot network connectivity with VPC Reachability Analyzer.



Maintain and modernize

- Modernize code with Amazon Q Developer Agent for code transformation.



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

39

Amazon Q supports developers across the software development lifecycle (SDLC) phases in several ways.

- **Plan:** You can use Amazon Q in the AWS Management Console to ask questions, receive best practices and recommendations, optimize Amazon Elastic Cloud Compute (EC2) instances, and learn more about the Well-Architected Framework. In the integrated development environment (IDE), Amazon Q can help you understand your code base and get up to speed on projects.
- **Create:** Amazon Q helps improve development productivity through in-line code recommendations directly in your IDE or command line interface (CLI), using natural language to generate new features. You can ask Amazon Q questions without leaving the IDE. The in-line code recommendations are available for multiple coding languages. For more information, see “Supported languages for Amazon Q Developer in the IDE” at <https://docs.aws.amazon.com/amazonq/latest/qdeveloper-ug/q-language-ide-support.html>.
- **Test and secure:** Amazon Q helps you confirm that your code is working and secure through unit test assistance. It helps you to find and fix security vulnerabilities earlier in the development cycle.
- **Operate:** Amazon Q can help troubleshoot errors, for example, in AWS Lambda, Amazon EC2, and Amazon Elastic Container Service (Amazon ECS). You can analyze your VPC reachability and get better debug and optimization tips.
- **Maintain and modernize:** Amazon Q Developer Agent for code transformation helps you maintain and modernize code by upgrading projects to more up-to-date language versions.

For more information, see “Accelerate your Software Development Lifecycle with Amazon Q” in the *AWS DevOps Blog* at <https://aws.amazon.com/blogs/devops/accelerate-your-software-development-lifecycle-with-amazon-q/>.

Example: Amazon Q Developer implements a new feature to a project – Prompt

The screenshot shows the Amazon Q Developer chat interface within an IDE. The interface includes a sidebar with various icons, a chat window titled 'AMAZON Q: CHAT', and a text input area. The chat window displays a welcome message from Amazon Q and a list of quick actions. The text input area contains a prompt: '/dev Create a RESTful API endpoint to expose the functionality through a web service'. Two annotations are present: a purple box labeled '1. Choose the Amazon Q icon.' pointing to the Amazon Q icon in the sidebar, and another purple box labeled '2. Send a prompt to Amazon Q.' pointing to the text input area.

1. Choose the Amazon Q icon.

2. Send a prompt to Amazon Q.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved. 40

In your integrated development environment (IDE), Amazon Q Developer can help you develop code features or make code changes to projects. You explain the feature that you want to develop, and Amazon Q uses the context of your current project to generate an implementation plan and the accompanying code to implement the feature. Amazon Q can help you build AWS projects or your own applications.

Example: Amazon Q Developer implements a new feature to a project – Plan

The screenshot shows the Amazon Q Developer chat interface. The chat window displays a plan for creating a RESTful API endpoint. The plan includes the following steps:

1. Add Servlet dependency
 - 1. Add the following dependency to enable Servlet support:
2. Create a REST resource class

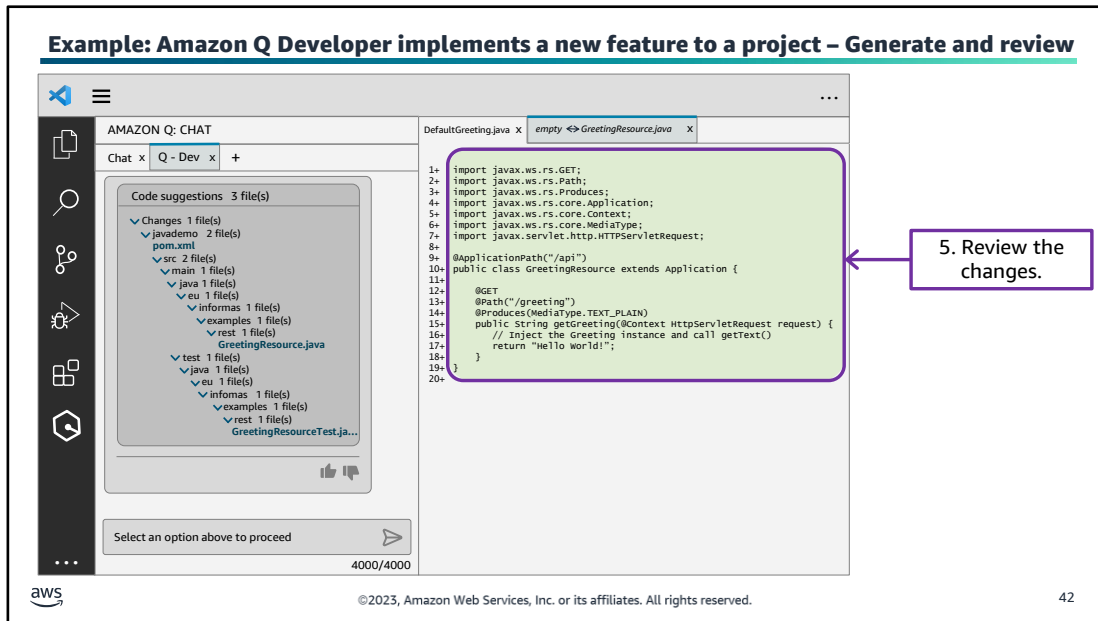
The plan also includes a note: "Note: To handle current demand, Q will use a smaller, higher-speed model for code generation."

Below the plan, there is a button labeled "Generate code" which is highlighted with a callout box labeled "4. Choose **Generate code**."

Another callout box labeled "3. Receive a plan created by Amazon Q." points to the plan content.

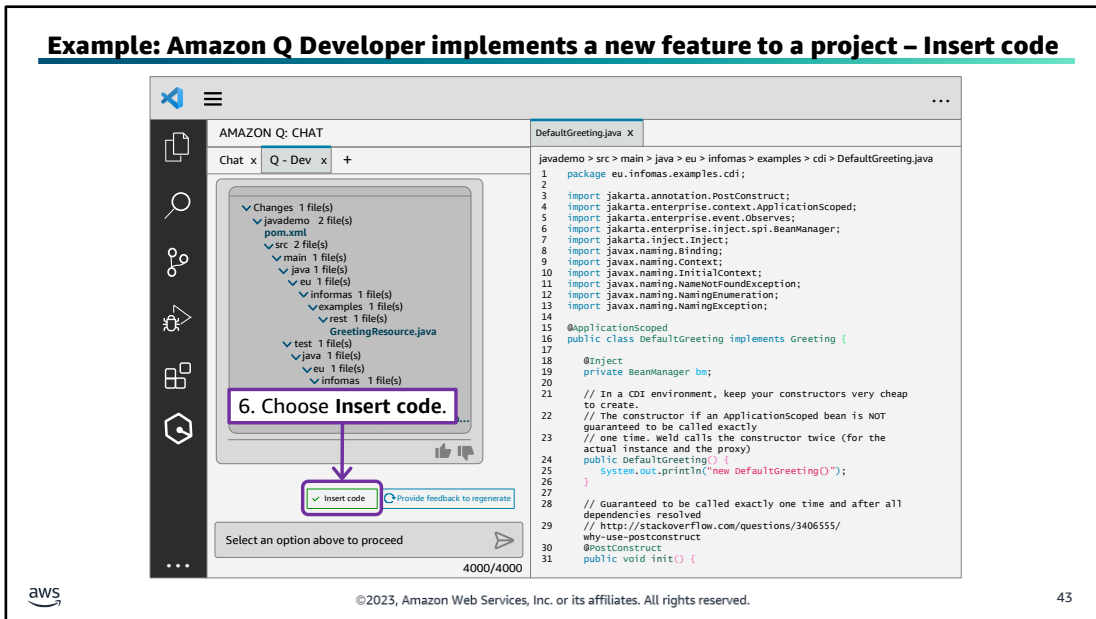
The interface also shows a sidebar with various icons, a search bar, and a footer with the AWS logo and copyright information: "©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved."

Amazon Q Developer analyzes the entire project and creates a plan for you to implement the requested task. It might take a few minutes depending on the size of the project. When the analysis is finished, the chat panel displays a step-by-step plan that details the changes to your code. Scroll through the steps to review them. Then, when you reach the end of the plan, you have the option to generate code. This is not the final step, and you will have an opportunity to review the changes before updating your code base.



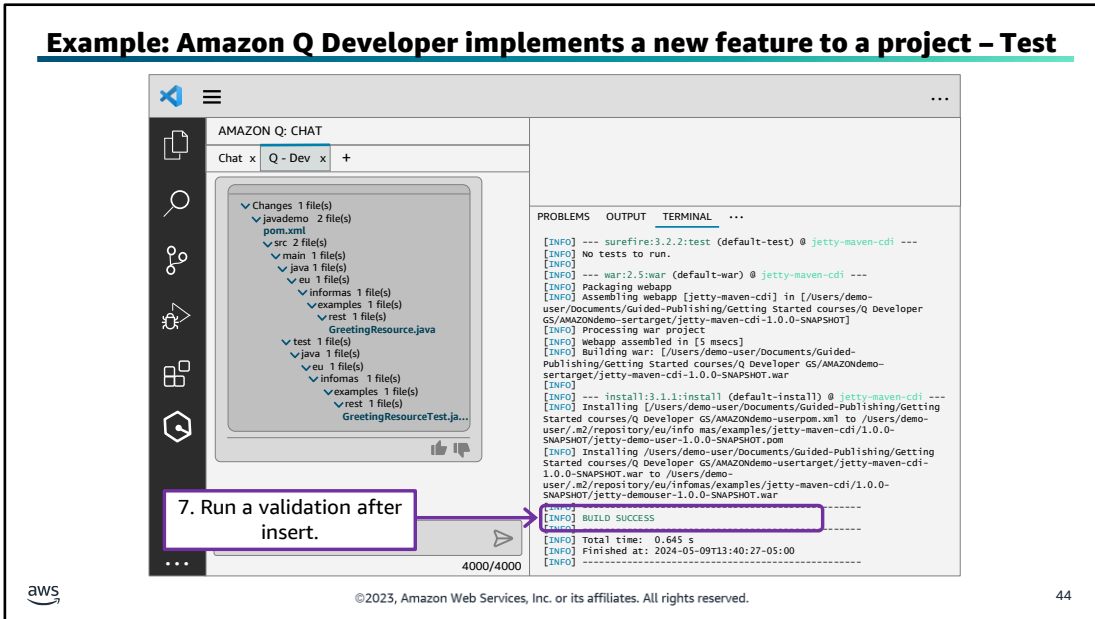
Amazon Q Developer generates code and proposes changes to add new features to your application by modifying existing files or creating new ones. Your codebase is represented in a tree structure. Modified or newly added files are displayed at the end of their respective folder paths. In the diff view, removed code is highlighted in red, while added code is highlighted in green. After reviewing the transformation summary, you can examine the specific changes Amazon Q is proposing in a file-by-file diff view. When Amazon Q Developer creates a new file, such as `GreetingResource.java`, that didn't exist in your original codebase, it's handled differently in the diff view. The diff view compares the new file to an empty file, displaying all the new code as additions.

Example: Amazon Q Developer implements a new feature to a project – Insert code



Scroll to the end of the chat contents and you have the option to choose **Insert code**. You can also regenerate code if you do not like the current suggestions. After the code update is complete, you can close the session.

Example: Amazon Q Developer implements a new feature to a project – Test



Finally, you can run a validation to verify that the new changes did not negatively affect the build. At the end of a clean install process, the terminal displays BUILD SUCCESS. The first tab displays the file tree and the second tab displays the code showing BUILD SUCCESS. Your code is ready for you to commit and deploy to your environment for testing.



It's now time to review the module and wrap up with a knowledge check.

Module summary

In this module, you learned how to do the following:

- Recognize the systems development lifecycle (SDLC).
- Describe how to start developing on AWS.
- Identify the benefits of using an integrated development environment (IDE).
- Indicate how to use AWS SDKs.
- Identify the benefits of using Amazon Q Developer.



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

46

In summary, in this module, you learned how to:

- Recognize the systems development lifecycle (SDLC).
- Describe how to start developing on AWS.
- Identify the benefits of using an integrated development environment (IDE).
- Indicate how to use AWS SDKs.
- Identify the benefits of using Amazon Q Developer.

To finish this module, complete the corresponding knowledge check.

Complete the knowledge check



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

47

It is now time to complete the knowledge check for this module.

Sample exam question

A developer needs a code assistant to help write code for a new application. The developer does not have a lot of experience with coding or with AWS technologies.

Which tool will meet this use case?

Identify the key words and phrases before continuing.

The following are the key words and phrases:

- Integrated development environment (IDE)
- Review their code and leave comments in real time from any computer that they have access to



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

48

It is important to fully understand the scenario and question being asked before even reading the answer choices. Find the keywords in this scenario and question that will help you find the correct answer.

Sample exam question: Response choices

A developer needs a code assistant to help write code for a new application. The developer does not have a lot of experience with coding or with AWS technologies.

Which tool will meet this use case?

Choice	Response
A	Use the AWS CLI to write code, reference AWS documentation when needed.
B	Use an Github repository to retrieve pre-built code from libraries..
C	Use Amazon Q Developer to write the code and the chat feature when needed.
D	Use the AWS SDK to build the application code reference AWS white papers when needed.



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

49

Now that we have bolded the keywords in this scenario, let us look at the answers.

Sample exam question: Answer

The correct answer is C.

Choice	Response
A	Use the AWS CLI to write code, reference AWS documentation when needed.
B	Use an Github repository to retrieve pre-built code from libraries.
C	Use Amazon Q Developer to write the code and the chat feature when needed.
D	Use the AWS SDK to build the application code reference AWS white papers when needed.



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

50

The correct answer is **C: Use Amazon Q Developer to write the code and the chat feature when needed.**

Additional resources

To learn more about the AWS SDK for Python used in this course:

- Developer guide – [Boto3 documentation](#)
- API documentation – [Core references](#)

To learn more about cloud development best practices:

- Well-Architected Framework, [Appendix: Questions and Best Practices](#) section



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

51

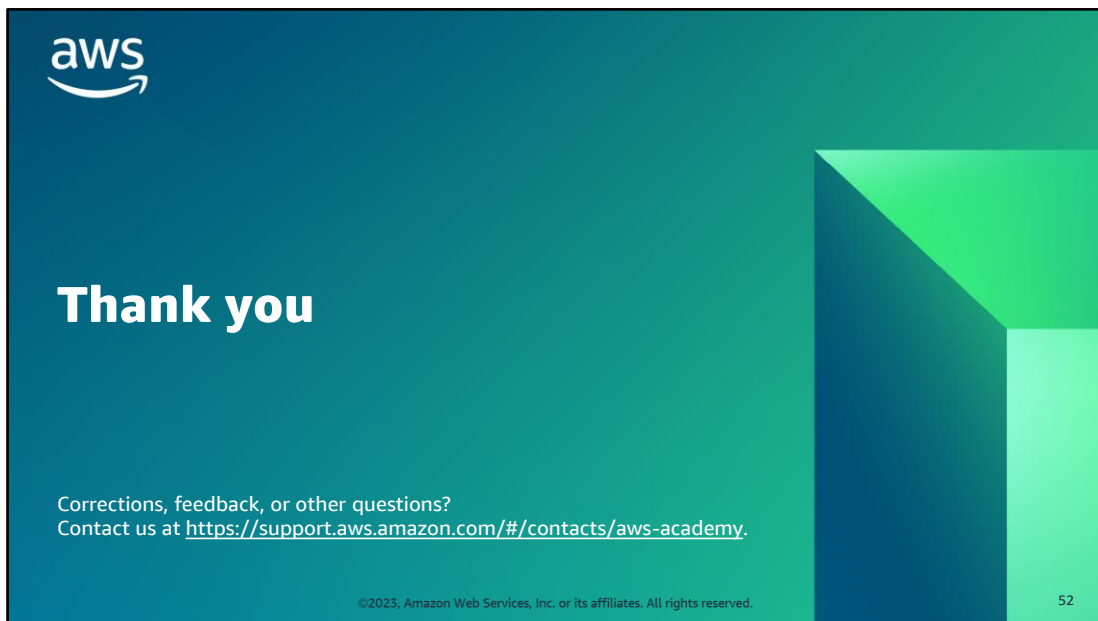
For all developer-related information, refer to the **AWS Tools & SDKs** link in the **Developer Tools** section on the AWS Documentation page (<http://aws.amazon.com/documentation/>).

For more information about the AWS SDKs, see:

- SDK for Java
 - Developer guide
<http://docs.aws.amazon.com/AWSSdkDocsJava/latest/DeveloperGuide/welcome.html>.
 - API documentation –
<http://docs.aws.amazon.com/AWSJavaSDK/latest/javadoc/index.html>.
- SDK for .NET and Xamarin
 - Developer guide – <https://docs.aws.amazon.com/sdk-for-net/v3/developer-guide/welcome.html>.
 - API documentation – <https://docs.aws.amazon.com/sdk-for-net/v3/developer-guide/welcome.html>.
- SDK for Python
 - Developer guide –
<https://boto3.amazonaws.com/v1/documentation/api/latest/index.html>.
 - API documentation –
<https://boto3.amazonaws.com/v1/documentation/api/latest/reference/core/index.html>.

To learn about cloud development best practices refer to the Well-Architected Framework, Appendix: Questions and Best Practices section at

<https://docs.aws.amazon.com/wellarchitected/latest/framework/appendix.html>.



Thank you for completing this module.